

REPORT

Summary

This report is composed of an introduction, and description of the resource allocation graph (RAG) features including its process, resource, cyclic_detector, cyclic_checker functions.

Introduction

Deadlocking is found in most computing systems with processing capabilities, where we have processing elements sharing resources to perform their tasks. It entails a scenario whereby elements remain in a state of idleness requiring one to initiate an action to re-trigger certain actions to perform their processing functions. Since it involves a state of idleness it is undesirable for multiprocessing or multiprogramming systems which have high throughput needs. Several mechanisms are available to detect and prevent deadlock events for systems with performance needs that ought to stay efficient for the overall system objectives.

Deadlocks can occur in systems that exchange or share data or any form of resource like networking, computing systems, operating systems, etc.

The Deadlock Program

- Resource Allocation Graph (RAG) is a form of representation for resource allocation and deallocation for processes applicable in operating systems or OS design concepts. The deadlock detector program harnesses a directed graph to depict resource and process allocation and deallocation. And it can be used to provide information on deadlocking nodes (devices) in the system.
- The Deadlock program uses a token bit string extension feature to track resource allocation and deallocation to processes based on user input.
- It uses a cyclic checker function that instantiates the cyclic detect function to detect deadlock conditions in real time. It performs the deadlock detection by traversing two look-up data structures for the process-to-resource (proc_tab) and resource-to-process (rec_tab) respectively, while checking for the presence of cycles i.e. if a process or resource appears twice in a separately maintained cyclic list for deadlock detection.
- The program accepts user input in the format name (input.txt) which can be specified in the program and outputs log messages in an output0.txt file.
- The program is written using Python.

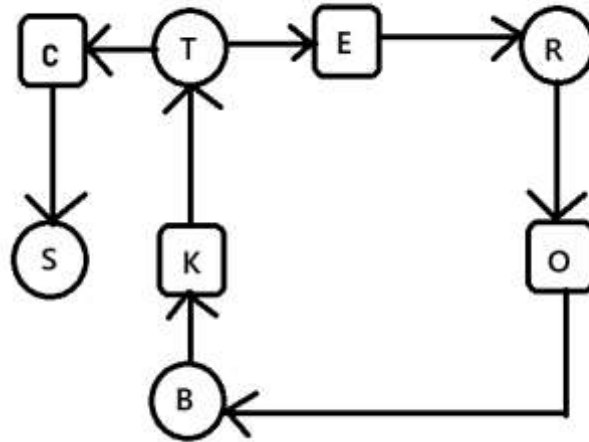


Figure 1. An example of a Resource Allocation Graph with processes T, R, B and resources E, O, K deadlocking or in a deadlock state.

- The deadlock_detector program consists of the following modules (class) and submodules:
 - Process
 - set_identifier, and add_proc functions
 - Resource
 - set_identifier, and add_rec functions
 - RAG
 - allocate
 - proc_num_gen, rec_num_gen, and proc_alloc_gen
 - deallocate
 - recproc_tabgen, and procrec_tabgen
 - cyclic_detect and cyclic_checker
- **The Process Module**
 - Process class provides the means to initialize an object and add processes to a list of existing processes using it add_proc function.
 - It also assigns a randomly generated process number (unique identifier) for all processes which are auto incremented in 10000 units when size of processes increases (initial set to 10000).
 - It maintains a list of all created processes with their process numbers (both user and system generated).
- **The Resource Module**
 - The Resource Module is similar to the Process class, but it encapsulates details for the systems resource management rather than its processes.

- It uses a generated alphanumeric ‘number’ to identify resources which is auto incremented in 10000 units with starting value of 20000.
- The Resource module keeps a list of all resources with the resource numbers (both user and system generated).
- **The RAG Module**
 - The RAG module consists of the following instance objects, functions and uses scenarios:
 - Instance Objects:
 - rec_proc instance – maintains a list of resource-to-process pairs using their unique numbers. Processes that are currently holding resources have their unique numbers bit extended with a ‘token’ bit string.
 - Proc_rec instance - maintains a list of process-to-resource pairs using their unique numbers.
 - proc instance – instance of the Process class.
 - rec instance – instance of the Resource class.
 - alloc_bit instance – tracks resource allocation status.
 - Sub-module or Functions
 - allocate – takes user provided process and resource number and allocates resources to processes. Also updates the proc_rec, rec_proc, process list, and resource list instances.
 - proc_num_gen - returns the generated unique process number.
 - rec_num_gen - returns the generated unique resource number.
 - proc_alloc_gen – returns the next process to be allocated resource.
 - deallocate - takes user provided process and resource number and deallocates resources and processes. Also it updates the proc_rec, rec_proc, process list, and resource list instances. It assigns a free resource to the next process.
 - recproc_tabgen – creates a look-up of resource-to-process dictionary data structure pairs which is used by the ‘cyclic_detect’ function to detect deadlock conditions. Resources are mapped to assigned processes holding resources.
 - procrec_tabgen – creates a look-up process-to-resource dictionary data structure pairs which is used by the ‘cyclic_detect’ function to detect deadlock conditions. Processes are mapped to resources they have ‘need’ for.
 - cyclic_detect – detects deadlock conditions in the RAG using lookup structure (from the recproc_tabgen – rec_tab and procrec_tabgen – proc tab

functions), current node, and an initialized list passed as parameters by the cyclic_checker function. It performs successive lookups in the rec_tab and proc_tab lookup structures to detect cycles in the RAG for deadlock detection. It returns a list of process and resource nodes that are deadlocked in the RAG.

- cyclic_checker – calls the cyclic_detect function with the following parameters; each node (process and resource in the RAG) with an attached proc_tab, rec_tab, and an initialized cyclic list. Also it outputs deadlock conditions, and if any with the corresponding processes and resources in an output log file.

- **The Main Module**

- The main module reads in a user provided input in the 'input0.txt' file in the format: {number} {N/R} {number}. The first number signifies a process number, next, N stands for 'need' or allocation request, 'R' for 'release' or deallocation request and the last number for a resource number.
- This module calls the RAG allocate, deallocate functions based on the user input and the cyclic_checker function prior and after input stream passing for deadlock detection.

- **Logging (output log file)**

- The Deadlock program uses Python logging module to implement logging functionality.
- The program outputs event statuses including deadlock conditions in the RAG during runs in the form of info messages in an output.log file.